21. Distributed Message Queue (Kafka-style)

> **Interviewer:** Design a distributed message queue — think Kafka. Producers publish messages, consumers read them, and it needs to handle **massive throughput durably**. Not "use a queue in your design" — actually **build the queue itself**, the thing other systems would depend on.

Candidate: Good — a different flavor than "design a service with users." Here *I* am the infrastructure. The bar is: never lose an acknowledged message, sustain huge sequential throughput, and let consumers replay history. Let me clarify first.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Many topics, or design for one at a time? 2. Ordering — global across a topic, or is per-key/per-partition order enough? Global order across a firehose usually kills parallelism. 3. Delivery guarantee — at-least-once with an idempotent consumer, or true exactly-once? 4. Replay — how far back? 5. Single-region to start, or multi-region/DR from day one?
>
> **Interviewer:** Many topics, millions of messages/sec in aggregate. Ordering only needs to hold **within a partition** — expected, not a limitation. Default **at-least-once**, but I want to hear how you'd reach exactly-once. Retention: **7 days**. Start single-region.

Candidate: Requirements, then.

Functional - `produce(topic, key?, value)` → durably appended, returns an offset. - `consume(topic, consumerGroup)` → `poll()` returns batches, ordered *within a partition*. - Consumers **commit** their read position and can **replay** by rewinding it. - Retention: 7 days, then eligible for deletion.

Non-functional - **Throughput over latency** — a write-optimized system; the design bends toward making appends as fast as physically possible. - **Durability**: once acknowledged, a message survives a machine dying. - **Ordering**: per-partition only, never topic-wide. - **Horizontal scalability**: more brokers → more partitions and throughput, near-linearly. - **Decoupling**: producers and consumers never call each other directly, don't need to be online at the same time.

---

2. Estimation

```
1M messages/sec across all topics, avg message size ~1 KB
  write throughput  ~= 1M * 1KB = ~1 GB/sec of raw data to persist

Retention: 7 days
  storage ~= 1 GB/s * 86,400 s/day * 7 days ~= ~600 TB

-> 600 TB and 1 GB/s can't live on one disk or one machine's NIC.
   The topic MUST be split and spread across many brokers.

Reads: several consumer GROUPS often read the same topic independently
  (alerting, analytics, archiving) -> read volume can exceed write
  volume, but reads are sequential and mostly recent -> cheap (Sec. 9).
```

Candidate: The core tension: **too much data for one disk, too much throughput for one machine.** Partitioning, the log format, and replication all exist to answer that.

---

3. A naive V1 (and why it caps out fast)

> **Interviewer:** Before going distributed — why not just a DB table? `INSERT` a row per message, consumers `SELECT ... WHERE id > last_seen`.

```
  [ Producer ] --INSERT--> [ messages table (Postgres) ] <--SELECT-- [ Consumer ]
                             id | topic | payload | consumed_by
```

Candidate: Every insert updates indexes, pays MVCC/transaction bookkeeping, and may take locks — none of which we need, since we never update or randomly access old rows. At 1M/sec that per-row overhead is the bottleneck: a general-purpose DB is built for arbitrary random-access read/write/delete, and we're paying for machinery we don't use. One table on one primary is also **one write bottleneck**, with no natural way to shard it without basically building a partitioned log ourselves.

The insight: a queue does exactly one kind of write (**append**) and mostly one kind of read (**read forward from a position**) — both map onto **sequential disk I/O**. Design only for that access pattern and skip almost all of a database's overhead: **replace "table" with "append-only log," and shard the log.**

---

4. Partitioning — splitting the topic, and the log itself

> **Interviewer:** Sell me on the log. What does it look like on disk, and how is it sharded?

Candidate: A **topic** (`"orders"`) splits into **partitions** — each partition is one ordered, append-only file living entirely on one machine (a **broker**). Every message gets an **offset**: its position in that log, 0, 1, 2, ... — a line number in an ever-growing file.

```
Topic "orders" — split into 3 partitions, spread across 3 brokers

  Partition 0 (Broker A):  [ msg@0 | msg@1 | msg@2 | msg@3 | ... ]  <- growing
  Partition 1 (Broker B):  [ msg@0 | msg@1 | msg@2 | ...          ]
  Partition 2 (Broker C):  [ msg@0 | msg@1 | ...                  ]

One message on disk:
------------------------------------------------
 offset     (int)     position within THIS partition
 key        (bytes?)  optional — picks the partition
 value      (bytes)   the payload
 timestamp  (long)
------------------------------------------------
```

Why this beats one giant log: a single append-only file is already fast, but capped by one disk's write bandwidth and one NIC. N partitions across N brokers means N disks/NICs share the load — the horizontal-scaling lever for writes.

Picking a partition: with a **key** (e.g., `customerId`), hash it → always the same partition — this is how you get "all events for customer 42, in order," since they land in one log appended by one writer. Without a key, round-robin for even load.

Why ordering is per-partition only: partitions are independent files appended by independent processes on different machines, no shared clock or lock. Global order would mean serializing every write through one place — exactly the bottleneck partitioning exists to kill.

```
Traditional DB write:   find the row -> lock it -> update in place
                         (random I/O + index updates + txn bookkeeping)

Log write (this design): append raw bytes to the end of the file
                         (sequential I/O — disk head never jumps,
                          no index to maintain on the write path)
```

Say-it-out-loud: *sequential writes to an append-only file can be an order of magnitude faster than random writes, because the disk never has to seek — every write is "append to where I already am."*

---

5. Replication — surviving a broker dying

> **Interviewer:** Each partition lives on one broker. That broker dies. What happens?

Candidate: Each partition replicates to a small set of brokers — one **leader** (all reads/writes) and **followers** that continuously pull and append the same log.

```
Partition 0, replication factor = 3

  Broker A: [ LEADER   ]  <- producers & consumers talk to THIS one
  Broker B: [ follower ]  <- continuously copies A's log, byte for byte
  Broker C: [ follower ]  <- continuously copies A's log

  ISR (in-sync replicas) = { A, B, C }   <- all caught up right now
```

The **ISR** (in-sync replicas) is the set of followers fully caught up with the leader — only ISR members are safe to promote; a stale follower promoted to leader could serve stale data or drop already-acked messages. **`acks` is the durability/latency dial** per write:

```
acks=0    : don't wait for anyone. Fastest; a crash can lose the
            message before it's written anywhere.
acks=1    : wait for the LEADER to write it. Safe against most
            failures, but lost if the leader dies before any
            follower copies it.
acks=all  : wait until every ISR member has it. Slowest, but the
            message survives even if the leader dies right after.
```

A payments topic runs `acks=all`; a clickstream topic runs `acks=1` for speed.

Who elects the leader and tracks metadata? A dedicated **coordination service** — historically **Zookeeper**, or **KRaft** (Kafka's built-in Raft) today. It keeps the authoritative "which broker leads which partition" map, detects a dead broker via heartbeats, and runs leader election among the ISR. We don't build this ourselves — we lean on a proven consensus layer for the one piece that truly needs strong, agreed consistency (who owns what), and keep it entirely out of the high-throughput data path.

---

6. Multi-region — mirroring, not synchronous replication

> **Interviewer:** Go multi-region — survive an entire region going dark.

Candidate: Not synchronous cross-region replication — an `acks=all` write would then need a round trip to another continent, on every produce call. Instead:

```
Region US-East                          Region EU-West
------------------                      ------------------
Cluster: topic "orders"    ---->        Cluster: topic "orders" (mirror)
  leader + followers                      leader + followers
  ALL within this region                  ALL within this region
       ^
       |  a MIRRORING job: a consumer in US-East that is also a
       |  producer into EU-West, async, decoupled from live writes
```

Each region runs its **own independent cluster**, all local, so the fast `acks=all` path never leaves the region. A separate **mirroring** process (Kafka's MirrorMaker) reads the source topic like any consumer and re-produces into the destination cluster on its own schedule. Trade-off: this is asynchronous — if US-East vanishes mid-write, whatever hadn't mirrored yet is lost. That is the accepted DR cost for keeping regional writes fast.

---

7. Latency and throughput — where the time goes

> **Interviewer:** Where does latency come from, and how do you claw it back?

Candidate: The biggest lever is **batching**:

```
Naive:   1 message -> 1 network write -> 1 round trip    (slow at scale)
Batched: hold messages a few ms (or until N bytes)
         -> send as ONE bigger network write             (fast)
```

The client buffers outgoing messages briefly, then ships a batch over a **long-lived TCP connection**, opened once and reused for thousands of messages — amortizing fixed per-request overhead across many messages, trading a few ms of latency for a large throughput multiple.

Why not REST? Per-call overhead (headers, connection setup, JSON) is fine at low volume, ruinous at millions/sec. A **custom binary protocol over persistent TCP** (Kafka's wire protocol) avoids it. And every latency knob here is really a durability knob in disguise: `acks=all` costs round trips; dropping to `acks=1` buys latency back at the cost of safety.

---

8. Consistency — delivery semantics, explained precisely

> **Interviewer:** What does "at-least-once" mean concretely, and how do you reach exactly-once for a payments topic?

```
At-most-once:   fire and forget. A crash before the ack lands means
                a SILENT loss. Rarely acceptable.

At-least-once:  (the default) if unsure a write/read landed, RETRY.
                Never silently loses a message, but the SAME message
                can arrive/process twice -> consumer must be
                idempotent (dedupe by message ID before acting).

Exactly-once:   needs BOTH:
                 1. Producer idempotence — producer ID + a monotonic
                    sequence number per partition; broker drops a
                    duplicate sequence number it already accepted.
                 2. The consumer's "read input, produce output, commit
                    offset" wrapped in ONE atomic transaction.
```

Duplicates happen because an **ack** can be lost even when the write itself succeeded — the producer retries blindly, and without idempotence the broker now has it twice. This is a real dial: a clickstream topic accepts at-least-once with cheap idempotent consumers; a payments topic pays exactly-once's transactional overhead because a duplicate charge is an incident.

---

9. Async by design, and what stays synchronous

> **Interviewer:** This whole system IS the async layer other services use. Inside the broker itself, what's synchronous and what's deferred?

Candidate: On **produce**, the only synchronous part is: append to the leader's log, and (per `acks`) wait for replicas to catch up — no downstream consumer's speed ever blocks a producer. On **consume**, a slow or crashed group simply falls behind — its **lag** (latest offset minus committed offset) grows, but never backs up onto producers or unrelated consumer groups; each group tracks its own offset independently. Systems built on top of this queue get "async" for free just by writing to a topic and consuming later — which only works because the queue guarantees durability before it acks.

---

10. Caching — the page cache does the job for free

> **Interviewer:** You're claiming huge read throughput off disk. Where's the cache layer?

Candidate: There isn't a bespoke one, deliberately.

```
PRODUCE:  producer -> batches -> leader appends to log file on disk
            -> leader replicates to followers -> once ISR has it, ACK

CONSUME:  consumer -> poll("messages after offset X")
            -> broker reads FORWARD from offset X on disk
            -> returns a batch -> consumer commits "I'm at X+N"
```

Consumers are usually near the tail of the log — reading data written moments ago, which is still sitting in the OS's **page cache** (RAM caching disk pages automatically, no app code needed). So a "disk read" for a live consumer is often actually a RAM read — a free consequence of the sequential, mostly-recent access pattern from Section 4.

---

11. Concurrency — consumer groups divide the work, zero locking

> **Interviewer:** Multiple consumers on one topic — how do you split work without contention?

Candidate: A **consumer group** is a team reading one topic; the queue assigns **each partition to exactly one consumer within a group**.

```
Topic "orders": 6 partitions.  Consumer group "billing": 3 consumers.

  Consumer 1 <- Partition 0, Partition 1
  Consumer 2 <- Partition 2, Partition 3
  Consumer 3 <- Partition 4, Partition 5

  -> zero contention: nothing is SHARED, so nothing to lock.
```

Ordering falls out naturally — one consumer per partition, appended in strict order, so that consumer sees them in order; across partitions there's no such guarantee. **Multiple groups** read the same topic fully independently, each tracking its own offsets. Offsets themselves are stored **durably** (an internal topic), so a restarted consumer resumes exactly where it left off.

---

12. Failure modes

Failure	What happens	Mitigation
Broker holding a leader dies	Partition stalls momentarily	ZK/KRaft promotes an ISR follower; clients refresh metadata and reconnect — no loss, replica was caught up
A follower falls behind	Drops out of the ISR	Leader serves from remaining ISR; `acks=all` waits on the smaller-but-safe set; rejoins once caught up
A consumer in a group dies	Group misses its heartbeat	Group **rebalances** partitions to live consumers, resumes from last **committed** offset; uncommitted work may be redelivered (at-least-once)
Consumer lag grows (slow, not dead)	Falls behind real-time	Monitor lag as a health metric; scale consumers up to the partition count, or speed processing
Coordination layer degrades	Elections/metadata stall	Small Raft/ZAB quorum tolerates minority loss; data path on known leaders keeps running
A whole region goes down	Cluster unreachable	Other regions unaffected; mirrored copy can be promoted to primary, accepting async lag as loss
Disk fills up	Broker can't append	Retention bounds growth; alert early, add brokers/disks, or shorten retention

The pattern to say out loud: replication makes an individual **broker's** death a non-event; consumer-group rebalancing makes an individual **consumer's** death a non-event — two separate mechanisms, storage side and reading side, neither depending on the other.

13. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Storage engine	Append-only commit log on local disk, not a general-purpose DB	Only appends and forward scans map to sequential I/O, far faster than the random I/O + index/lock overhead a DB (Postgres/MySQL) pays per row. A DB-backed queue (`INSERT` / `SELECT WHERE id > x`) caps out long before 1M msg/s.
Sharding unit	Partitions within a topic, each pinned to one broker	Spreads throughput/storage across many disks/NICs — the horizontal-scaling lever for writes; the cost is ordering only per-partition, since global order would re-serialize all writers through one place.
Durability / replication	Leader + follower replicas per partition, ISR-gated election, tunable `acks`	A real latency/safety dial (`acks=0/1/all`); promoting only from the ISR (not any replica) stops a stale follower from silently serving missing data after failover.
Coordination (leader/metadata)	Dedicated consensus layer — Zookeeper, or KRaft	Leader election and partition ownership need strong, agreed consistency among a few nodes — a Raft/ZAB problem, isolated from the data path so the hot path never pays consensus cost per message. Rolling this into brokers directly

Concern	Choice	Why this, and why not the alternative
		risks split-brain — two brokers both believing they lead the same partition.
Client-broker routing	Smart clients with cached metadata, no classic load balancer	A partition's data lives on one specific leader; a generic LB can't route per-partition traffic without itself knowing ownership. Clients fetch a small "who leads what" map and connect directly.
Wire protocol	Custom binary protocol over long-lived TCP, client-side batching	REST's per-call overhead (connections, headers, JSON) is ruinous at millions/sec; persistent binary connections let the client batch and amortize overhead.
Read acceleration	OS page cache, no bespoke read cache	Consumers read near the tail; recently-written data is already in RAM via normal OS disk-page caching — a free win from the sequential-access design.
Idempotence / exactly-once	Producer ID + sequence numbers, plus transactional read-produce-commit on consume	Turns "retry on ambiguous ack" from unsafe into safe; full exactly-once needs read+write+offset-commit as one atomic unit — expensive, so opt-in per topic, not the default.

Say-it-out-loud justification: "*The queue's own storage is an append-only log, not a database, because we only append and read forward — sequential I/O beats a general-purpose DB's random-access, lock-and-index overhead by an order of magnitude at this throughput. Partitioning spreads that log across brokers for horizontal write scale, at the cost of only guaranteeing order per-partition. Leader/follower replication with ISR-gated failover and a tunable* `acks` *gives a real durability/latency dial. A dedicated consensus layer — Zookeeper or KRaft — handles the one thing that truly needs strong consistency, who leads what, off the hot path. And at-least-once is the pragmatic default, with idempotent producers/consumers as the safety net, because exactly-once's transactional overhead is only worth paying where duplicates are truly unacceptable, like money movement.*"

---

14. Follow-up curveballs

> **Interviewer:** How do you choose the number of partitions up front?

Candidate: A balance, not a max. More partitions means more parallelism, but each carries overhead — file handles, replication traffic, slower elections/rebalances as the count grows. Size for peak throughput divided by per-partition throughput (benchmark it), add headroom, and remember **partition count caps how much one consumer group can parallelize** — extra consumers beyond that just sit idle.

> **Interviewer:** One key gets far more traffic than any other — a "hot partition." Now what?

Candidate: That key always hashes to the same partition (Section 4), so it's the bottleneck regardless of how many other partitions exist — more partitions elsewhere don't help. Fix: pick a better key (shard-

suffix a hot entity across several partitions, accepting weaker per-entity ordering), or give that hot entity dedicated capacity.

> **Interviewer:** Why not drop retention to zero once every consumer has read a message?

Candidate: That kills replay, a hard requirement — a new consumer group tomorrow (a new pipeline, a backfill) couldn't see yesterday's data. Retention is **time-based, not consumption-based** — the log doesn't track who's "finished" reading; it ages data out after a fixed window, which is exactly what makes "replay the last N days" possible.

> **Interviewer:** Two producers write to the same partition at nearly the same instant — how do you avoid losing one to a race?

Candidate: No race at the log level — the partition leader is the single writer; both requests hit the same leader process, which appends them one after another, never concurrently at the same file position. There's nothing shared to race over because there's exactly one writer; whichever request the network delivered first is appended first.

---

15. Deep-dive: why not just poll a database table instead of an append-only log?

> **Interviewer:** Section 3 waved away "just use a Postgres table" pretty fast. Be precise — mechanically, where does a DB-table queue lose to the log?

Candidate: Three separate costs stack up, and none of them is really "disk is slow" — a DB's disk is just as fast. It's that the table forces work the log never has to do.

1. Polling overhead — there's no "push," only "ask again"

A DB row has no way to notify anyone. A consumer must ask on a timer: `SELECT id, payload FROM messages WHERE status='pending' ORDER BY id LIMIT 100`.

```
Tight poll (e.g. every 50ms):
  ask -> empty -> ask -> empty -> ask -> empty -> ask -> FOUND ONE
  most round trips do real query planning + index lookups for ZERO rows

Loose poll (e.g. every 2s):
  ask -> empty -> ... -> ask -> FOUND ONE
  fewer wasted queries, but every message now waits up to 2s to be seen
```

There's no setting that avoids the trade-off, because polling is fundamentally "ask and hope." A broker consumer instead issues a **fetch the broker can hold open** (long-poll) until data exists or a timeout passes — zero wasted round trips *and* low latency, because the broker tells the consumer the instant something lands instead of the consumer guessing an interval.

2. Lock contention on the claim

With more than one worker, someone has to make sure two workers don't grab the same row — that's the **claim**: `SELECT ... FOR UPDATE SKIP LOCKED`, or an `UPDATE ... SET status= 'claimed', worker_id=? WHERE id = (SELECT id ... LIMIT 1 FOR UPDATE SKIP LOCKED)`. Under real concurrency this hurts twice: workers contend for row-level locks on the same "front" of the table (even `SKIP LOCKED` still has to scan past whatever's currently held), and marking a row processed is an `UPDATE`, not a read — every message now leaves a dead tuple behind (Postgres MVCC), which `VACUUM` has to reclaim, and the `status` index has to be maintained *live* on every single message, forever. The log has no equivalent claim step at all: a partition is handed to exactly one consumer **once**, at the group level (Section 11), not fought over message-by-message — so steady-state consumption takes zero locks.

3. Sequential append + offset consumption

The log turns both sides of the workload into exactly what a disk is fastest at: writes are pure appends (no index to update mid-write), reads are "give me everything after offset X," a forward scan the OS page cache usually already holds in RAM (Section 10). The DB table's writes *look* like appends but aren't only that — the secondary index on `status` / `id` that polling depends on needs random-order maintenance on every insert and every claim `UPDATE`. And consumption needs **no shared mutable state**: an offset is a private integer a consumer tracks, nothing another consumer can race against — versus the DB's `status` column, which is shared mutable state by definition, the very thing the claim step exists to protect.

Concern	DB-table-as-queue	Append-only-log broker
New-message discovery	Poll on an interval — a latency/waste trade-off with no free setting	Long-poll fetch, pushed the instant data lands
Concurrent consumers	Row-level locks / `SKIP LOCKED` on the claim; contention rises with worker count	Partition assigned once per group; zero per-message locking
Marking progress	An `UPDATE` per message → MVCC bloat, vacuum, live index maintenance	A private offset commit; no row state mutated
Write path	Insert + index maintenance (random I/O in disguise)	Pure sequential append
Read path	Indexed lookup for `WHERE status='pending'`	Sequential forward scan from an offset
Scaling lever	Bigger primary, more indexes, more partitioning you build yourself	More partitions/brokers, near-linear
Ceiling	Thousands/sec before lock waits and vacuum dominate	Millions/sec, gated by disk/NIC bandwidth

> **Rule of thumb:** *If consumers have to poll and lock rows to find work, you've rebuilt a queue on top of a general-purpose database's transactional machinery — every message pays for indexing, locking, and MVCC bookkeeping it doesn't need. That's fine at a few hundred messages/sec with a handful of workers; past that, pay for an append-only log instead, where "find work" is a sequential scan and "claim work" doesn't exist as an operation.*

16. Deep-dive: the crash-before-commit edge case — at-least-once vs at-most-once, and rebalancing

> **Interviewer:** Section 8 named the delivery semantics. Now walk me through the actual failure: a consumer processes a message, then dies before it commits the offset. What happens on restart — and how do you keep that from being either a silent duplicate or a silent loss?

Candidate: This is the sharpest edge case in the whole system, and it comes down to **commit ordering** — *when* you commit the offset relative to *when* you do the work. There's no way to make both "commit" and "do the side effect" atomic for free, so whichever one you do second is the one that can be skipped by a crash.

Timeline 1 — commit AFTER processing (default: at-least-once → possible duplicate)

```
t0  consumer polls  -> gets message @ offset 100 (e.g. "charge card $50")
t1  consumer PROCESSES it -> charge succeeds, card is charged
t2  consumer CRASHES  -- before calling commitOffset(101)
--- restart, or partition reassigned to another consumer in the group ---
t3  last committed offset is still 100 (never advanced)
t4  broker redelivers offset 100 -- "not yet committed, from your point of view"
t5  consumer PROCESSES it again -> charge succeeds AGAIN -> customer charged twice
```

Timeline 2 — commit BEFORE processing (at-most-once → possible loss)

```
t0  consumer polls  -> gets message @ offset 100
t1  consumer commits offset 101 IMMEDIATELY (before doing any work)
t2  consumer CRASHES  -- before actually charging the card
--- restart ---
t3  last committed offset is 101 -- broker believes 100 is done
t4  offset 100 is NEVER redelivered to anyone
t5  the charge silently never happens -- no error, no retry, just gone
```

The two timelines are mirror images: whichever of {commit, process} happens second is the one a crash can erase. You cannot make an in-process side effect and a remote offset commit a single atomic unit without extra machinery — which is exactly what the fixes below buy, at increasing cost.

Ranked fixes

1. **At-least-once + idempotent consumer — the practical default.** Always process before committing (Timeline 1's ordering, never Timeline 2's — a duplicate is recoverable, a silent loss is not). Make reprocessing harmless: dedupe on a message/idempotency key (a unique constraint, or a "processed IDs" cache with a TTL past max redelivery time), or make the side effect naturally idempotent — `UPSERT` instead of `INSERT`, pass an idempotency key to the downstream payment API so a retried charge is recognized and ignored.
2. **Transactional / exactly-once semantics — for where duplicates are truly unacceptable.** Kafka's read-process-write transactions tie "read offset X, produce output Y, commit offset X" into one atomic unit via a transactional producer and `read_committed` isolation on downstream

consumers (Section 8). Cost: extra coordinator round trips and transaction markers hurt throughput, and it only spans Kafka-to-Kafka — if the side effect is a REST call to an external payment gateway, the transaction's atomicity stops at Kafka's edge; that external call still needs its own idempotency key regardless.

3. **Offset-commit ordering as a lever on its own.** Manual commit right after the side effect completes narrows the crash window to "the instant between finishing work and one commit call." Periodic **auto-commit** is worse by default: it can fire on a timer mid-batch and commit offsets for messages the consumer hasn't actually finished — silently sliding toward Timeline 2's loss profile without anyone intending it.

The same symptom from a different trigger: consumer-group rebalancing

Redelivery isn't only a crash story. When a consumer misses a heartbeat — slow GC pause, OOM, a rolling deploy killing the pod — the group coordinator reassigns its partitions to the remaining members (Section 11's mechanism, this is its cost):

```
Before:  Consumer A owns partition 2, has processed up to offset 150,
         but only COMMITTED up to offset 148 (149–150 in flight)

A misses its heartbeat -> group REBALANCES -> partition 2 -> Consumer B

After:   Consumer B resumes reading from the last COMMITTED offset: 148
         -> messages 149 and 150 are redelivered to B, even though A
            already fully processed them before it stalled
```

Same duplicate-processing outcome as Timeline 1, but the trigger is group membership churn, not a crash — which is why "idempotent consumer" has to hold regardless of *why* redelivery happened. Aggressive (eager) rebalancing that stops every consumer in the group during reassignment makes this worse by pausing the whole group at once; incremental/cooperative rebalancing (Kafka's default since 2.4) limits disruption to only the reassigned partitions.

Recommended approach

At-least-once with idempotent consumers as the default posture — process, *then* commit, *then* make reprocessing a no-op via a dedupe key. Reserve transactional exactly-once for the narrow set of topics where a duplicate is an incident, not an inconvenience (payments, billing ledgers), accepting its throughput cost there deliberately. Keep per-message processing short and idempotent so a rebalance-triggered redelivery — which *will* happen — costs nothing more than redoing a cheap, safe operation.

> **What to say out loud:** "*Whichever of 'commit the offset' or 'do the side effect' happens second is the one a crash can erase — commit-after-process risks a duplicate, commit- before-process risks a silent loss, and a silent loss is the one I never accept. So the default is at-least-once with an idempotent consumer: process first, commit after, and dedupe on a message key so redelivery — whether from a crash or a group rebalance — is a safe no-op. I'd only pay for transactional exactly-once on topics where a duplicate is itself an incident, like money movement, because that atomicity only spans Kafka-to-Kafka and still needs an idempotency key the moment a side effect leaves the cluster.*"

Summary — the mental model

A distributed message queue is **"build the append-only log everything else leans on"** — not a service with users, but the durable, ordered, replayable backbone underneath other services. The winning moves: split each topic into **partitions**, each an **append-only log** turning every write into sequential disk I/O instead of a database's random-access overhead; **replicate** each partition leader→followers with an ISR and tunable `acks` so a broker dying is a non-event; hand leader election and metadata to a dedicated **Zookeeper/KRaft** consensus layer kept off the hot path; let **consumer groups** divide partitions with zero contention; get read acceleration for free from the **OS page cache**; and default to **at-least-once** with idempotent producers/consumers, reserving full exactly-once transactions for where duplicates are truly unacceptable. Every latency knob here — `acks`, batching window, exactly-once — is really a durability knob wearing a latency costume.